

7.4 Counting

Settings

Beta features

Counting with a while loop

Counting is a common task in programming, often done using a loop. A **loop variable** can be used for iteration until the loop condition is False. Ex: If a while loop iterates 10 times, the loop variable can be incremented inside the loop body to count the iterations.

A common error is to forget to update the loop variable at the end of the loop (Ex: `loop_variable` unintended infinite loop).

PARTICIPATION ACTIVITY

7.4.1: Basic counting with a while loop.

Start ☐ 2x speed

```
loop_variable = 1

while loop_variable <= 5:
    print(loop_variable)
    loop_variable = loop_variable + 1
```

6 loop

Output

1
2
3
4
5

Captions ^

1. The program begins by initializing loop_variable with 1.
2. Since loop_variable is 1 (which is <= 5), the first iteration begins, and 1 is printed. Then, loop_variable is incremented to 2.
3. In the second iteration, 2 is printed. Then, loop_variable is incremented to 3.
4. In the next iterations, 3, 4, 5 are printed. Once loop_variable is 6, the loop condition is False, and the loop ends. The output shows 5 iterations counted.

**PARTICIPATION
ACTIVITY**

7.4.2: Basic loop iteration counting.

1) How many times does the following loop iterate?

```
count = 1

while count <= 99:
    print(count)
    count = count + 1
```

- ☐ 98
- ☐ 99
- ☐ 100

2) How many times does the following loop iterate?

```
count = 1

while count < 10:
    print(count)
    count = count + 1
```

- ☐ 9
- ☐ 10
- ☐ 11

3) How many times does the following loop iterate?

```
count = 0

while count < 50:
    print(count)
    count = count + 1
```

- ☐ 48
- ☐ 49
- ☐ 50

4) What happens in the following code?

```
count = 1

while count <= 2:
    print(count)
count = count + 1
```

- ☐ The loop iterates only once and prints 1. Then, count is incremented after the loop ends.
- ☐ The loop prints 1 for two iterations. Then, count is incremented after the loop ends.
- ☐ The loop does not end and prints 1 continuously. Count is not incremented.

Variations on counting with a while loop

A variation on counting with a while loop is to decrement the loop variable to count down. Another variation is to count in steps greater than 1. Also, the loop variable can be modified using other arithmetic operations, including multiplication and division.

**PARTICIPATION
ACTIVITY**

7.4.3: Counting down with a while loop.

Start

2x speed

```
variation = 10

while variation >= 0:
    print(variation)
    variation -= 5
```

-5 varia

Output

10
5
0

Captions ^

1. The program begins by initializing variation with 10.
2. Since variation is ≥ 0 , the first iteration begins, and 10 is printed. Then, variation is decremented by 5 (shorthand for `variation = variation - 5`).
3. In the second iteration, 5 is printed, and variation is decremented to 0.
4. In the third iteration, 0 is printed. After decrementing, variation is -5 (which is < 0), and the loop ends. The program is counting down to 0 in steps of 5.

Other compound operators

- **`variation += 5`** increments variation by 5 (that is, **`variation = variation + 5`**).
- **`variation *= 5`** multiplies variation by 5 (that is, **`variation = variation * 5`**).
- **`variation /= 5`** divides variation by 5 (that is, **`variation = variation / 5`**).

When changing how the loop variable is updated, the variable's initialization and the loop condition must be changed to start, count, and end as expected.

**PARTICIPATION
ACTIVITY**

7.4.4: Variations on counting.

1) What happens in the following loop?

```
num = 9

while num > 0:
    print(num)
    num -= 2
```

- ☐ The loop outputs all integers between 9 and 1 in descending order.
- ☐ The loop outputs odd integers between 9 and 1 in descending order.
- ☐ The loop outputs odd integers between 9 and 1 in descending order and then outputs 0 at the end.

2) What happens in the following loop?

```
num = -10

while num <= 10:
    print(num)
    num += 5
```

- ☐ The loop outputs integers from 0 to 10 in steps of 5.
- ☐ The loop outputs integers from -10 to 5 in steps of 5.
- ☐ The loop outputs integers from -10 to 10 in steps of 5.

3) For the following loop to print 1, 10, 100, what compound operator is needed in the blank?

```
num = 1

while num <= 100:
    print(num)
    num ____ 10
```

- ☐ +=
- ☐ =*
- ☐ *=

**PARTICIPATION
ACTIVITY**

7.4.5: Counting in a loop simulator.

The following tool allows you to enter values for a loop's parts, and then executes the loop. Using problem individually.

The tool can use relational operators $<$, $<=$, $>$, and $>=$. Equality ($=$, $!=$), identity (is , $is\ not$), and membership.

1. 0 to 10,000 by 500s (0, 500, 1000, ...)
2. -19 to 19 by 1s
3. 10 to -10 by 1s
4. Multiples of 3 between 0 and 100
5. Powers of 2 from 1 to 256 (1, 2, 4, 8, ...)
6. Come up with your own challenges

Applications of counting with a while loop

Counting with a while loop has applications for mathematics and sequence calculations. Ex: A loop programming the following:

- N factorial ($N!$) is the product of all positive integers less than or equal to N (that is, $N! = N * (N-1) * (N-2) * \dots * 1$)
- The Fibonacci sequence starts with 0 or 1, and each subsequent integer is the sum of the two preceding ones
- A savings interest program can calculate the yearly savings with interest (Ex: `savings += savings * interest`)

**PARTICIPATION
ACTIVITY**

7.4.6: Factorial calculation.

Start ☐ 2x speed

```
factorial = 4
count = factorial - 1

print("count")

while count >= 1:
    print(" ", count)
    factorial *= count
    count -= 1

print("Factorial: 4! =", factorial)
```

24

0

Output

```
count
3
2
1
Factorial: 4! =
```

Captions ^

1. The program begins by initializing factorial with 4 and count with 3, which is factorial - 1. is printed.
2. count is ≥ 1 , so the first iteration begins and 3 is printed. Then, factorial is calculated as 2.
3. count is still ≥ 1 , so the second iteration begins and 2 is printed. Then, factorial is calculated and decremented to 1.
4. count is still ≥ 1 , so the third iteration begins and 1 is printed. Then, factorial is calculated and decremented to 0.
5. Since count is 0 and no longer ≥ 1 , the loop ends. The calculated factorial, $4! = 24$, is printed.

**PARTICIPATION
ACTIVITY**

7.4.7: Applying while loop counting.

1) What happens when the factorial loop condition changes from `count >= 1` to `count > 1`, as shown in the following?

```
factorial = 4
count = factorial - 1

print("count")

while count > 1:
    print(" ", count)
    factorial *= count
    count -= 1

print("Factorial: 4! =", factorial)
```

- ☐ The count output and calculated factorial remain the same.
- ☐ The factorial is incorrectly calculated as 12 due to one less iteration of the loop.
- ☐ Only 3 and 2 are output for count, but the calculated factorial remains the same.

2) What is the output of the following?

```
count = 5
first = 1
second = 1
print(first)
print(second)

while count > 0:
    sequence = first + second
    print(sequence)
    first = second
    second = sequence
    count -= 1
```

- ☐ 5, 4, 3, 2, 1
- ☐ 0, 1, 1, 2, 3, 5, 8
- ☐ 1, 1, 2, 3, 5, 8, 13

3) What happens in the following loop?

```
count = 0

while count <= 100:
    if count % 2 == 0:
        print(count)
    count += 1
```

- ☐ The loop outputs even integers between 0 and 100, not including 0 and 100.
- ☐ The loop outputs even integers starting with 0 and ending with 100.
- ☐ The loop outputs odd integers between 0 and 100.

**CHALLENGE
ACTIVITY**

7.4.1: Loops with variables that count.

763650.1722066.qx3zqy7

**CHALLENGE
ACTIVITY**

7.4.2: Counting.

763650.1722066.qx3zqy7

Start

Integer `user_input` is read from input. Integer `i` is initialized with 21. Complete the while loop to each iteration until `i` is less than `user_input`:

- Decrement `i`.
- If `i` is divisible by 3, then output the value of `i`.

[Click here for example](#) ▼

Note: `i % 3 == 0` returns True if `i` is divisible by 3.

```
1 user_input = int(input())
2 i = 21
3
4 while i >= user_input:
5
6     """ Your code goes here """
7
```

1

Check

Next level

View solution ▼ *(Instructors only)*